How Far Can 0.6B Parameters Go? Compute-Efficient Generative Question Answering with LoRA Fine-Tuning

Author One **Author Two** **Author Three** **Author Four**

Author Five
EN.737 Large Language Models — course project
`https://github.com/Syndr0/compute-efficient-genqa`

Abstract

We study how much compute a small generative language model needs to become a competent context-grounded question answerer, whether broader training data buys out-of-domain robustness, and how to serve the result efficiently. Using Qwen3-0.6B, LoRA, and the MRQA benchmark collection under a single controlled pipeline, we find that (i) adaptation is extremely cheap: 250k training tokens (578 questions, 20 seconds on one GPU) raise mean F1 from 41.4 (3-shot prompting) to 70.6 in-domain and from 34.3 to 62.6 on unseen BioASQ and DuoRC, and every further $4\times$ increase in budget adds only about 2 F1 up to 16M tokens; (ii) at a fixed 4M-token budget, mixing SQuAD, NewsQA and TriviaQA does *not* improve out-of-domain F1 over SQuAD-only training (paired differences -0.33 and $+0.49$ F1, both 95% CIs include zero over three seeds), contrary to our pre-registered hypothesis, while it costs 1.2 F1 on SQuAD itself; and (iii) static batching raises inference throughput $7.1\times$ (to 116 questions/s) at batch size 64 before it declines, at the cost of $9\times$ higher latency, while length-sorting the batch nearly doubles throughput at large batch sizes and quality is unchanged throughout. We release code, configurations, logs and per-question predictions.

1 Introduction

Small generative language models are attractive for question answering (QA) over a provided passage: they run on a single commodity GPU, can be adapted cheaply, and produce free-form answers without a task-specific span-extraction head. How much compute such adaptation needs, how well the adapted model transfers to unseen domains, and how to serve it efficiently are practical questions that are rarely answered together under one controlled setup.

We study them with **Qwen3-0.6B** [17] adapted by low-rank adaptation (LoRA) [9] to context-grounded single-turn QA on the MRQA collection [6]. All experiments share one model, one prompt, one LoRA configuration and one evaluation pipeline, and the training data are controlled by an exact *training-token budget*. We ask:

- **RQ1** (training efficiency): how does QA quality change with the LoRA training-token budget, relative to zero- and few-shot prompting?
- **RQ2** (generalization): at a fixed budget, does multi-source fine-tuning (SQuAD + NewsQA + TriviaQA) generalize better to unseen domains (BioASQ, DuoRC) than SQuAD-only fine-tuning?

Preprint.

- **RQ3** (inference efficiency): how does the inference batch size trade throughput against latency and GPU memory?

Hypotheses and quantitative predictions were fixed before the main runs; Appendix C compares them with the outcomes.

Contributions. (1) A controlled, fully reproducible pipeline for token-budgeted LoRA fine-tuning and evaluation of a small generative QA model, including passage-level splits, overlap filtering, paired-bootstrap statistics and error analysis. (2) Evidence that the step from prompting to *any* fine-tuning dominates the returns: 0.25M tokens recover most of the attainable F1 and further budget yields a roughly log-linear $\approx$2 F1 per $4\times$ tokens. (3) A negative result on data diversity: at equal token budgets, multi-source training does not improve out-of-domain F1 for this model, and per-source validation loss is a misleading proxy for cross-domain answer quality. (4) An inference characterisation showing an interior throughput optimum for static batching and the large effect of length-sorting.

2 Related work

Parameter-efficient adaptation. LoRA [9] trains low-rank updates to frozen weight matrices and is the default way to adapt LLMs on limited hardware; QLoRA [3] additionally quantizes the frozen base. A small amount of supervised data can suffice to change a model's output behaviour [26]; we quantify this for context-grounded QA in units of training tokens and GPU time.

Generalization in reading comprehension. The MRQA shared task [6] standardised six training and six held-out QA datasets to measure out-of-domain transfer. Training on several sources improved the robustness of extractive models [21, 6], and multi-format training benefits generative QA [12, 18]. Such comparisons usually confound diversity with data quantity; we hold the token budget fixed. Domain shift and continued adaptation are discussed by Gururangan et al. [7].

Compute budgets and inference. Scaling studies relate loss to parameters and training tokens [11, 8]. Autoregressive decoding is memory-bound, so batching amortizes weight reads [16]; serving systems such as Orca [25] and vLLM [14] schedule requests dynamically and IO-aware kernels speed up attention [2]. We measure the basic static-batching trade-off these systems build on.

3 Experimental setup

Data. We use the MRQA 2019 versions [6] of SQuAD [19], NewsQA [22] and TriviaQA [10] (web) for training and in-domain (ID) evaluation, and BioASQ [23] and DuoRC (ParaphraseRC) [20] as out-of-domain (OOD) test sets that are never used for training or model selection. Training pools are capped at 50k questions per source. From each pool we hold out whole passages (300 questions) as a development set used only for validation loss. ID test sets come from the MRQA validation splits and OOD sets from the MRQA test split. Eval questions whose passage also occurs in a training pool (after whitespace and case normalisation) are removed before sampling 1,000 questions per set; this removed 552 NewsQA and 601 TriviaQA candidates (Table 5 in Appendix A). MRQA markup tokens are converted to plain text.

Task format and model. Each input is a single chat turn: a system instruction asking for the shortest answer span copied from the passage, followed by `Passage: ... Question: ...`. Qwen3's thinking mode is disabled and passages are truncated to 768 tokens (at test time the head of the passage is kept; for training the window is moved to contain the answer). Few-shot prompting prepends three fixed short SQuAD training examples as earlier chat turns [1]. Answers are decoded greedily (at most 32 tokens) and the first line is kept.

LoRA fine-tuning under a token budget. Adapters of rank 16 (α=32, dropout 0.05) are attached to all attention and MLP projections: 10.1M trainable parameters, 1.7% of the 606M total. We use AdamW, learning rate 2×10^{-4}, 5% warm-up and cosine decay, 16 sequences per optimizer step and the loss on answer tokens only. A budget B means that examples are drawn without

Table 1: **Prompting vs. LoRA at increasing training-token budgets** (F1, 1,000 questions per set; three-source mixture, seed 0). GPU-min is training time; dev loss is the mean answer-token loss on the three held-out dev sets. Best per column in bold. 95% bootstrap CIs are about ±2–3 F1 (Appendix B).

			In-domain			Out-of-domain		Mean	
System	GPU-min	Dev loss	SQuAD	NewsQA	TriviaQA	BioASQ	DuoRC	ID	OOD
Zero-shot	–	–	33.2	21.7	17.1	23.8	15.2	24.0	19.5
3-shot	–	–	55.5	31.5	37.2	41.1	27.5	41.4	34.3
LoRA 250k	0.3	0.844	82.3	56.5	73.0	69.9	55.3	70.6	62.6
LoRA 1M	1.3	0.662	86.7	57.4	70.0	72.3	55.7	71.3	64.0
LoRA 4M	5.0	0.563	88.4	63.1	72.7	**74.3**	58.5	74.7	66.4
LoRA 16M	20.1	0.476	**91.0**	**67.9**	**76.0**	74.2	**59.8**	**78.3**	**67.0**

replacement—with an equal token share per source—until their prompt plus answer tokens sum to B; the model makes a single pass with a schedule fitted to that length. Cost is reported as wall-clock GPU time and estimated FLOPs $\approx 4NT$ for N parameters and T tokens (forward plus activation gradients; the frozen base needs no weight gradients). **RQ1** trains on the three-source mixture with $B \in \{0.25, 1, 4, 16\}$M tokens. **RQ2** compares SQuAD-only training with the three-source mixture at B=4M over three seeds (data sample and adapter initialisation); the mixture's seed-0 run is shared with **RQ1**.

Evaluation and statistics. We report Exact Match (EM) and token-level F1 with the official SQuAD/MRQA normalisation, maximised over gold answers. Confidence intervals are 95% percentile bootstrap intervals over questions [5]. System comparisons use a paired bootstrap over the same 1,000 questions [13] with 10,000 resamples, applied to per-question scores averaged over seeds; we also report the seed-to-seed standard deviation [4].

Inference benchmark. **RQ3** decodes 512 SQuAD prompts with the merged 16M-token adapter in bf16 using HuggingFace `generate` [24]: batch sizes 1–256 with length-sorted, left-padded batches and the KV cache on, plus ablations with unsorted batches, the KV cache disabled (64-prompt subset) and fp16/fp32 weights. Each configuration has one warm-up batch and three timed repeats (run-to-run standard deviation below 0.3% of throughput). Latency is the wall-clock time of the batch that contains a request.

Hardware. All final runs used one NVIDIA RTX PRO 6000 Blackwell Server Edition GPU (96 GB; Google Colab "G4") with PyTorch, Transformers [24] and PEFT [15]. All LoRA training for this paper took 48 GPU-minutes.

4 Results

4.1 RQ1: training efficiency

Table 1 and Figure 1 show that the largest gain comes from the first, smallest budget. Zero-shot, the model answers in full sentences (EM 2–8%); three demonstrations raise mean F1 to 41.4 ID and 34.3 OOD. Fine-tuning on only 250k tokens—578 questions and 20 seconds of training—lifts mean F1 to 70.6 ID (+29.2) and 62.6 OOD (+28.3), with gains on every set (SQuAD +26.8, BioASQ +28.8, DuoRC +27.8). Beyond that, returns are modest and roughly log-linear: averaged over the five sets, F1 rises from 67.4 (250k) to 68.4 (1M), 71.4 (4M) and 73.8 (16M), i.e. about 2 F1 per $4\times$ more tokens, so the $64\times$ larger 16M budget adds 7.7 ID and 4.4 OOD points for $60\times$ the compute. The OOD curve flattens earlier than the ID curve: BioASQ peaks at 4M (74.3 vs. 74.2 at 16M), whereas NewsQA, which has the longest answers, keeps improving ($56.5 \rightarrow 67.9$).

Validation loss decreases monotonically with the budget (0.84, 0.66, 0.56, 0.48 nats per answer token; 3.23 before training) and orders the runs like mean ID F1, but not every set: TriviaQA F1 dips at 1M (70.0 vs. 73.0 at 250k) although its dev loss falls. Training throughput is constant at 12.7–13.3k tokens/s across budgets (Table 7), so cost is linear in tokens: the 16M run took 20 minutes and 3.9×10^{16} FLOPs (Figure 2).

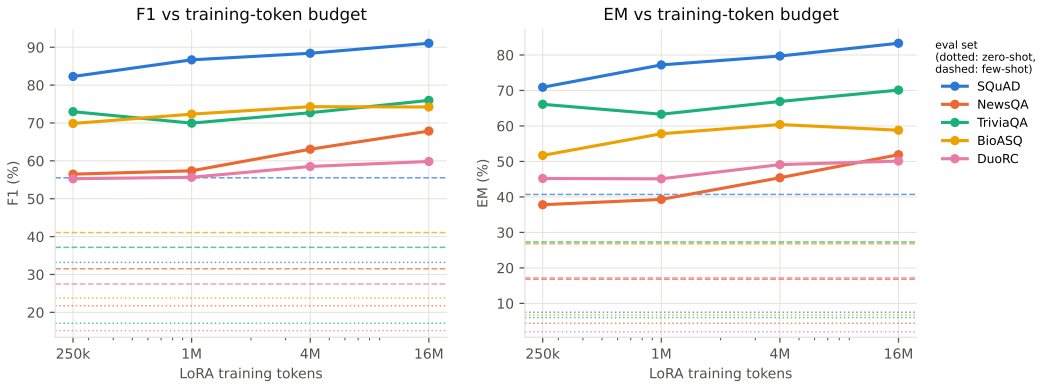


Figure 1: **F1 and EM vs. LoRA training tokens** (log scale). Dotted and dashed horizontal lines are the zero- and 3-shot baselines of the same colour.

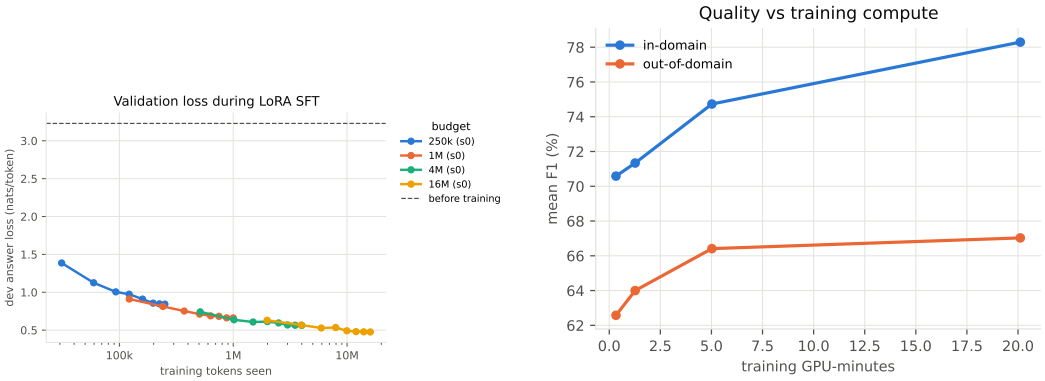


Figure 2: **Left:** validation answer loss during training for each budget. **Right:** mean ID and OOD F1 against training GPU-minutes.

4.2 RQ2: does multi-source training generalize better?

Table 2 answers **RQ2** negatively for this model and budget. On the two unseen domains the multi-source mixture is statistically indistinguishable from SQuAD-only training: ΔF1 $= -0.33$ [-1.40, +0.77] on BioASQ and $+0.49$ [-0.80, +1.79] on DuoRC, and the mean OOD F1 is 66.2 for both conditions. The seed-to-seed standard deviation (0.6–1.2 F1) is as large as the differences, and individual seeds point both ways (Appendix B): on DuoRC, one seed favours the mixture by $+2.4$ F1 (p=0.01) and the other two favour SQuAD by about 0.5. Only a single-seed comparison would have suggested a significant effect. In domain, the mixture helps a little on the two sources it adds (NewsQA $+1.2$, p=0.07; TriviaQA $+1.0$, p=0.32) and costs 1.2 F1 on SQuAD [-2.07, -0.29] (p=0.008), the only significant difference.

Why no OOD gain? Two observations explain part of it. First, equal tokens are not equal supervision: SQuAD passages are short, so the SQuAD-only run sees 16.8k questions and 102k answer tokens, versus 9.2k questions and 54k answer tokens for the mixture, and it trains 38% faster (18.4k vs. 13.3k tokens/s). Second, the base model already reads well; what fine-tuning mostly teaches is the *answer format*—short spans copied from the passage—and SQuAD alone teaches it. The error analysis (Table 8) supports this: the mixture produces shorter answers (mean predicted length on TriviaQA 1.97 $\rightarrow$ 1.55 words, BioASQ 1.88 $\rightarrow$ 1.73) and fewer *verbose* answers (TriviaQA 9.9% $\rightarrow$ 3.9%, BioASQ 10.6% $\rightarrow$ 7.5%), but these mostly turn into *partial* matches rather than exact ones, and the dominant OOD failure—choosing the wrong span (13.6–13.8% on BioASQ, 27.8–28.2% on DuoRC)—is unchanged. A further 7% of DuoRC answers fall outside the 768-token window and are unanswerable as posed.

Table 2: **SQuAD-only (single) vs. SQuAD + NewsQA + TriviaQA (multi) at 4M training tokens.** Mean ± std over 3 seeds; ΔF1 is multi minus single with a 95% paired-bootstrap CI over questions (scores averaged over seeds) and a two-sided p-value.

		EM		F1		ΔF1 (multi − single)		
Eval set	Type	Single	Multi	Single	Multi	Δ	95% CI	p
SQuAD	ID	81.6 ± 0.2	80.5 ± 0.7	89.7 ± 0.2	88.5 ± 0.2	-1.17	[-2.07, -0.29]	0.008
NewsQA	ID	44.3 ± 0.8	45.3 ± 0.7	62.1 ± 0.9	63.3 ± 0.2	+1.21	[-0.10, +2.53]	0.068
TriviaQA	ID	65.9 ± 1.0	67.9 ± 1.2	72.8 ± 0.9	73.8 ± 1.1	+1.01	[-0.91, +2.95]	0.320
BioASQ	OOD	61.2 ± 0.8	60.1 ± 0.6	74.3 ± 0.7	73.9 ± 0.6	-0.33	[-1.40, +0.77]	0.547
DuoRC	OOD	49.3 ± 0.8	48.6 ± 1.7	58.0 ± 0.8	58.5 ± 1.2	+0.49	[-0.80, +1.79]	0.457

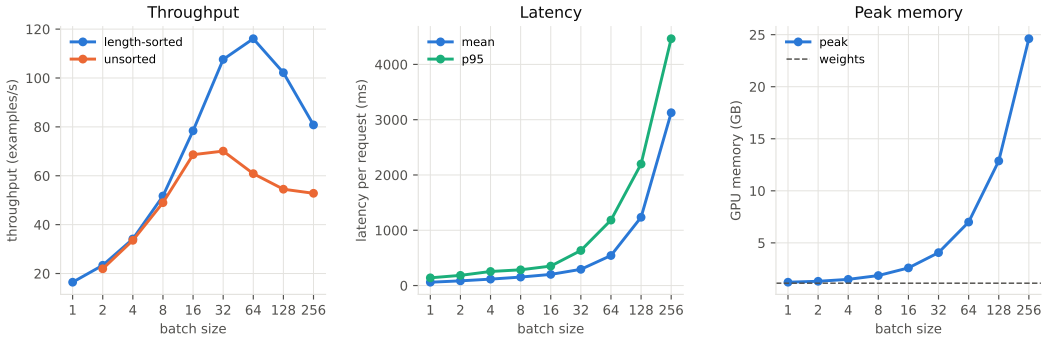


Figure 3: **Batch size vs. throughput, per-request latency and peak GPU memory** for the merged 16M-token adapter (bf16, 512 SQuAD prompts). The dashed line is the memory of the weights.

Validation loss is a poor cross-domain proxy. The SQuAD-only adapters have a mean dev loss of 1.25 versus 0.55 for the mixture, driven by a TriviaQA dev loss of 2.68 vs. 0.89. Yet their TriviaQA F1 is only 1.0 point lower. TriviaQA gold answers are lower-cased aliases; the SQuAD-only model assigns low probability to that surface form but still produces the right entity, which F1 normalisation forgives. Model selection by pooled dev loss would therefore strongly, and wrongly, favour the mixture.

4.3 RQ3: inference efficiency

Throughput grows from 16.4 questions/s at batch size 1 to a maximum of 116.1 at batch size 64 (7.1×) and then *declines* to 102 (128) and 81 (256) (Figure 3, Table 3). The decline has two causes visible in the logs: with static batching every sequence waits for the longest answer in its batch, and at large batch sizes the length spread inside a batch grows, so padding rises to 30–48% of prompt tokens even after sorting. Latency grows faster than throughput: a request waits 543 ms at batch 64 (9× batch 1) and 3.1 s at batch 256. Peak memory grows almost linearly with batch size, from 1.2 GB (weights 1.1 GB) to 7.0 GB at 64 and 24.6 GB at 256, dominated by the KV cache and prefill activations of 700-token prompts. Quality is unaffected: F1 stays within 90.3–90.9 across all batch sizes.

Length sorting is the cheapest optimisation: unsorted batches carry 46–65% padding at batch ≥ 16 and lose up to 48% of throughput (60.9 vs. 116.1 questions/s at batch 64), with their optimum shifting down to batch 32. Disabling the KV cache costs nothing at batch 1—answers are only about five tokens long, so decoding is dominated by the prefill—but is 1.7× slower at batch 8 and 5.1× slower at batch 32 (Table 4). fp16 matches bf16 in speed and quality, while fp32 doubles memory and is 1.4× slower at batch 32.

5 Discussion and limitations

For a 0.6B model on context-grounded QA, the practical lesson is that adaptation compute is not the bottleneck: a few hundred examples close most of the gap to a much longer run, and a 16M-token

Table 3: **Inference batch sweep** (length-sorted, KV cache on, bf16). Latency is per batch, i.e. per request. The unsorted column repeats the sweep with batches in random length order.

Batch	Ex/s	Gen tok/s	Latency (ms)	p95 (ms)	Peak mem (GB)	Padding (%)	Ex/s unsorted	F1
1	16.4	81	60	140	1.22	0.0	–	90.8
2	23.4	115	85	184	1.31	0.2	21.9	90.6
4	34.1	167	116	254	1.49	0.7	33.6	90.4
8	51.7	252	153	285	1.86	1.7	49.0	90.9
16	78.4	385	202	352	2.59	3.8	68.6	90.7
32	107.6	526	293	634	4.06	8.6	70.1	90.3
64	116.1	567	543	1184	7.00	16.9	60.9	90.5
128	102.2	501	1235	2198	12.87	29.9	54.5	90.6
256	80.8	396	3126	4465	24.61	47.5	52.8	90.6

Table 4: **Inference ablations.** KV-cache comparisons use a 64-prompt subset; precision comparisons use all 512 prompts. Speed-up is default (bf16, KV cache on) over the variant.

Variant vs. default	Batch	Ex/s (variant)	Ex/s (default)	Default speed-up	Peak mem (GB)
KV cache off vs. on	1	15.44	15.38	1.0×	1.14 / 1.19
KV cache off vs. on	8	26.39	45.49	1.7×	1.24 / 1.66
KV cache off vs. on	32	17.27	87.77	5.1×	1.59 / 3.26
float16 vs. bfloat16	1	16.50	16.44	1.0×	1.22 / 1.22
float16 vs. bfloat16	32	107.85	107.60	1.0×	4.06 / 4.06
float32 vs. bfloat16	1	15.52	16.44	1.1×	2.48 / 1.22
float32 vs. bfloat16	32	76.26	107.60	1.4×	8.09 / 4.06

run costs 20 GPU-minutes. Because the gain beyond the smallest budget is log-linear and smaller out of domain, a practitioner with a fixed budget would do better spending it on evaluation, seeds and data quality than on longer training. Our **RQ2** result cautions against assuming that data diversity transfers: at matched tokens, diversity traded answer-format calibration for in-domain accuracy without moving OOD F1, and with a single seed we would have drawn a different conclusion.

Limitations. We study one base model, one LoRA configuration and one prompt; larger models or other adapters may behave differently. Budgets up to 16M tokens are small compared with the full MRQA training sets, and diversity may matter more at larger budgets or with more distant sources. The equal-token design gives SQuAD-only training more questions and answer tokens, so "diversity at equal supervision" is not what **RQ2** measures. Each set has 1,000 questions, so differences of about one F1 point are at the resolution of our CIs; EM/F1 penalise correct paraphrases, and overlap filtering uses exact normalised matches only. **RQ3** uses HuggingFace `generate` with static batching on a single GPU type; serving engines with continuous batching and paged attention [14] would shift the throughput optimum. Finally, our hypotheses were pre-registered against a T4 configuration and the final runs used a larger GPU profile (Appendix C).

6 Conclusion

A 0.6B-parameter model becomes a strong extractive-style QA system with remarkably little fine-tuning: 250k LoRA training tokens add about 29 F1 over few-shot prompting both in and out of domain, and further budget yields about 2 F1 per $4\times$ tokens. Mixing source datasets at a fixed token budget did not improve transfer to biomedical or movie-plot questions, and pooled validation loss would have suggested otherwise. At inference, length-sorted static batching at batch size 32–64 gives a 6.5–$7\times$ throughput gain with sub-second latency and unchanged quality.

Reproducibility. Every number in this paper is produced by `python -m cegqa.cli all -config configs/default.yaml -config configs/a100.yaml` from the released repository, which also writes the tables and figures used here; `results/` contains the metrics, training logs, statistics and benchmark rows behind them. The notebooks in `notebooks/` run the same pipeline on Google Colab.

Author contributions. P1 built the data and evaluation pipeline (MRQA processing, passage-level splits, overlap checks, EM/F1 and prediction logging). P2 built the prompting baselines and the LoRA training pipeline with token-budget sampling and cost accounting (**RQ1**). P3 designed and analysed the generalization study, including the paired bootstrap and error analysis (**RQ2**). P4 built the inference benchmark (**RQ3**). P5 led the analysis, figures and writing. All authors interpreted the results and edited the paper.

References

[1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, 2020.

[2] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.

[3] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, 2023.

[4] Jesse Dodge, Gabriel Ilharco, Roy Schwartz, Ali Farhadi, Hannaneh Hajishirzi, and Noah Smith. Fine-tuning pretrained language models: Weight initializations, data orders, and early stopping. *arXiv preprint arXiv:2002.06305*, 2020.

[5] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993.

[6] Adam Fisch, Alon Talmor, Robin Jia, Minjoon Seo, Eunsol Choi, and Danqi Chen. MRQA 2019 shared task: Evaluating generalization in reading comprehension. In *Proceedings of the 2nd Workshop on Machine Reading for Question Answering*, 2019.

[7] Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. Don't stop pretraining: Adapt language models to domains and tasks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020.

[8] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, 2022.

[9] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[10] Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, 2017.

[11] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[12] Daniel Khashabi, Sewon Min, Tushar Khot, Ashish Sabharwal, Oyvind Tafjord, Peter Clark, and Hannaneh Hajishirzi. UnifiedQA: Crossing format boundaries with a single QA system. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020.

[13] Philipp Koehn. Statistical significance tests for machine translation evaluation. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, 2004.

[14] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.

[15] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. PEFT: State-of-the-art parameter-efficient fine-tuning methods. `https://github.com/huggingface/peft`, 2022.

[16] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems*, 2023.

[17] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[18] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140), 2020.

[19] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, 2016.

[20] Amrita Saha, Rahul Aralikatte, Mitesh M. Khapra, and Karthik Sankaranarayanan. DuoRC: Towards complex language understanding with paraphrased reading comprehension. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics*, 2018.

[21] Alon Talmor and Jonathan Berant. MultiQA: An empirical investigation of generalization and transfer in reading comprehension. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.

[22] Adam Trischler, Tong Wang, Xingdi Yuan, Justin Harris, Alessandro Sordoni, Philip Bachman, and Kaheer Suleman. NewsQA: A machine comprehension dataset. In *Proceedings of the 2nd Workshop on Representation Learning for NLP*, 2017.

[23] George Tsatsaronis, Georgios Balikas, Prodromos Malakasiotis, Ioannis Partalas, Matthias Zschunke, Michael R. Alvers, Dirk Weissenborn, Anastasia Krithara, Sergios Petridis, Dimitris Polychronopoulos, et al. An overview of the BIOASQ large-scale biomedical semantic indexing and question answering competition. *BMC Bioinformatics*, 16, 2015.

[24] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020.

[25] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation*, 2022.

[26] Chunting Zhou, Pengfei Liu, Puxin Xu, Srini Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, et al. LIMA: Less is more for alignment. In *Advances in Neural Information Processing Systems*, 2023.

A Data

Table 5: **Datasets after processing.** Train pool size, number of evaluation questions, mean passage and answer length (words) and the share of eval questions whose gold answer occurs in the passage.

Dataset	Split	Train pool	Eval	Passage words	Answer words	Answer in passage (%)
SQuAD	ID	49997	1000	125	3.0	100.0
NewsQA	ID	50000	1000	492	4.0	100.0
TriviaQA	ID	50000	1000	643	1.6	99.9
BioASQ	OOD	–	1000	208	2.0	97.8
DuoRC	OOD	–	1000	623	1.9	95.5

Before filtering, 552 of 4,212 NewsQA and 601 of 7,785 TriviaQA validation questions shared their passage with a training pool (the same source documents recur across MRQA splits); these were removed before sampling. After filtering no eval passage occurs in a training pool; 62 NewsQA questions still share their exact question text with a training question (generic questions such as "What is his age?").

B Additional results

Table 6: **RQ1 in Exact Match.** Same systems as Table 1.

			In-domain			Out-of-domain		Mean	
System	GPU-min	Dev loss	SQuAD	NewsQA	TriviaQA	BioASQ	DuoRC	ID	OOD
Zero-shot	–	–	7.5	4.4	6.0	6.7	2.0	6.0	4.3
3-shot	–	–	40.7	16.8	27.3	26.8	17.2	28.3	22.0
LoRA 250k	0.3	0.844	70.9	37.8	66.1	51.7	45.2	58.3	48.5
LoRA 1M	1.3	0.662	77.2	39.3	63.3	57.8	45.1	59.9	51.5
LoRA 4M	5.0	0.563	79.7	45.4	66.9	**60.4**	49.1	64.0	**54.8**
LoRA 16M	20.1	0.476	**83.3**	**51.9**	**70.1**	58.8	**50.1**	**68.4**	54.5

Table 7: **Training cost per budget** (three-source mixture, seed 0, one RTX PRO 6000).

Budget	Examples	Tokens seen	Tokens/s	GPU-min	Est. FLOPs	Peak mem (GB)
250k	578	250,853	12,713	0.3	6.08e+14	18.1
1M	2,300	1,000,947	13,166	1.3	2.43e+15	18.3
4M	9,169	4,000,398	13,277	5.0	9.70e+15	18.4
16M	36,620	16,000,613	13,260	20.1	3.88e+16	19.0

Table 8: **Error categories** (% of questions, seed 0, 4M tokens). *Verbose*: prediction contains a gold answer plus extra words; *wrong span*: F1 $= 0$ and the prediction occurs in the passage; *truncated*: the gold answer is not in the visible 768-token window.

Eval set	Train	Correct	Partial	Verbose	Wrong span	Not in passage	Empty	Truncated
SQuAD	single	81.6	6.2	7.2	4.8	0.2	0.0	0.0
	multi	79.7	7.0	6.7	6.3	0.3	0.0	0.0
NewsQA	single	44.1	20.2	11.7	22.9	0.3	0.0	0.8
	multi	45.4	20.4	10.4	22.9	0.1	0.0	0.8
TriviaQA	single	66.9	2.8	9.9	18.5	0.2	0.0	1.7
	multi	66.9	6.2	3.9	19.0	2.3	0.0	1.7
BioASQ	single	61.1	12.4	10.6	13.8	0.6	0.0	1.5
	multi	60.4	16.4	7.5	13.6	0.5	0.0	1.6
DuoRC	single	50.3	7.9	6.1	28.2	0.4	0.0	7.1
	multi	49.1	8.4	6.2	27.8	1.1	0.0	7.4

Per-seed paired differences (multi − single, F1). SQuAD -1.35, -1.15, -1.01; NewsQA $+1.07$, $+2.29$, $+0.27$; TriviaQA -1.09, $+3.04$, $+1.10$; BioASQ -0.32, -1.53, $+0.85$; DuoRC -0.41, -0.55, $+2.42$. The seed-averaged comparison in Table 2 is the pre-registered analysis.

C Pre-registered hypotheses and outcomes

Hypotheses were fixed in `docs/hypotheses.md` before the main runs, for a T4 configuration (budgets up to 4M, RQ2 at 1M, 500 questions per set). The final runs used the large-GPU profile (budgets up to 16M, RQ2 at 4M, 1,000 questions per set); the predictions are evaluated unchanged.

- **H1a** (250k tokens beat the best prompting baseline on every ID set; $\geq$15 F1 on SQuAD): *supported* ($+26.8$ F1 on SQuAD; $+25.0$ NewsQA; $+35.8$ TriviaQA).

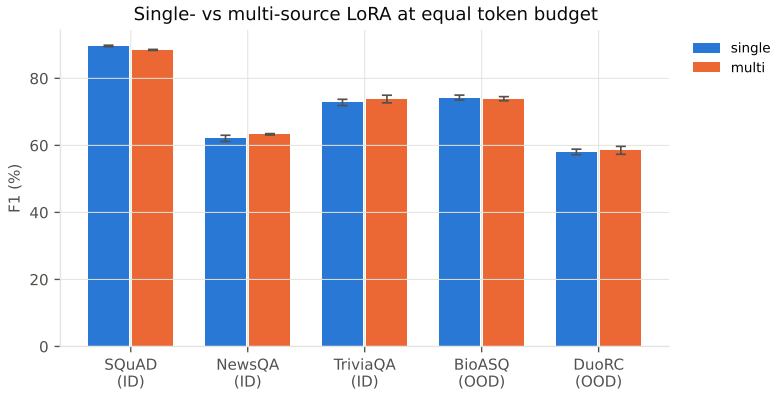


Figure 4: **RQ2 F1 by eval set**, mean and standard deviation over three seeds.

- **H1b** (diminishing returns; 1M→4M adds less than half of 0.25M→1M): *not supported*. Gains per $4\times$ budget are roughly constant (+1.0, +3.0, +2.4 mean F1), i.e. F1 is close to linear in log tokens, which is the functional form we predicted, but without the predicted decay.
- **H1c** (dev loss decreases monotonically and ranks runs like dev F1): *supported* for mean ID F1; TriviaQA alone is non-monotonic.
- **H1d** (tokens/s within $\pm 10\%$ across budgets): *supported* (12.7–13.3k tokens/s).
- **H2a** (multi-source improves mean OOD F1 by 2–5 points with a CI excluding 0 on at least one OOD set): *not supported* ($+0.08$ mean OOD F1; both CIs include 0).
- **H2b** (gain larger on DuoRC than BioASQ): directionally consistent ($+0.49$ vs. -0.33) but not significant.
- **H2c** (SQuAD-only within ± 2 F1 on SQuAD): *supported* (SQuAD-only better by 1.2).
- **H2d** (multi-source mainly reduces not-in-passage and wrong-span errors): *not supported*; it mainly reduces verbose answers, and wrong-span errors are unchanged.
- **H3a** ($\geq 8\times$ throughput, saturating at batch 32–64): *partly supported* ($7.1\times$, peak at 64).
- **H3b** (latency at batch 64 $\geq 3\times$ batch 1): *supported* ($9.0\times$).
- **H3c** (memory roughly linear in batch size, below 16 GB up to 64): *supported* (7.0 GB at 64).
- **H3d** (quality within ± 0.5 F1; unsorted $\geq 15\%$ slower at large batches; KV cache off $\geq 2\times$ slower): *supported* for quality (± 0.3) and sorting (48% slower at 64); *partly* for the KV cache ($5.1\times$ at batch 32, but no difference at batch 1 because answers are short).